REM Flight Cheat Sheets

REM Project

Edition 1

These sheets are quick lookups for the REM repository. The architecture manual and current target headers are authoritative when a detail changes.

1. Build with make

```
PREFIX ?= .toolchain-install/bin/myemulator2-elf-
AS      = $(PREFIX)as
LD      = $(PREFIX)ld
OBJDUMP = $(PREFIX)objdump

all: program.elf
program.elf: main.o output.o
    $(LD) -T toolchain/userspace/myemulator2-user.ld $^ -o $@
main.o: main.S
    $(AS) $< -o $@
output.o: output.S
    $(AS) $< -o $@
inspect: program.elf
    $(OBJDUMP) -dr $<
clean:
    rm -f *.o program.elf
.PHONY: all inspect clean
```

`make` is a dependency manager. It decides what needs rebuilding; `as` assembles and `ld` links.

```
make              build what is out of date
make inspect      disassemble and inspect relocations
make -B            rebuild everything
make clean         remove generated objects
```

Keep generated objects separate from source. A missing symbol is a module-boundary error; an assembler error is a source/syntax error; a linker-script error is an image-layout error.

2. GAS syntax and directives

```
.section .text
.global _start
.type _start, @function
_start:
        li   r1, 5
        add  r2, r1, r0
        ret
.size _start, .-_start

.section .rodata
message: .ascii "REM\n"
end:     .byte 0
```

```
Comments             # comment
Registers            r0 ... r15 (lowercase canonical form)
Memory               displacement(base), for example 0(r5)
Alignment            .align 2      # target alignment convention
Data                 .byte .short .word .ascii .asciz .space
Symbols              label: .equ NAME, value
Visibility            .global .extern
Function metadata    .type name,@function  .size name,.-name
Addresses            %hi(symbol), %lo(symbol)
```

Use the REM assembly manual for instruction operands and encodings. Use the GNU `as` reference extract for directives, expressions, sections, and relocations.

3. REM function ABI

```
r0       hardwired constant zero
r1-r4    first four ordinary function arguments
r1       ordinary 32-bit return value
r13 / sp downward-growing stack pointer
r14 / lr return link register
r5-r12   callee-saved registers
r15      caller-saved temporary
```

```
Call boundary: stack is 16-byte aligned
Red zone:      none
Nested call:   save r14 before jal/call
Return:        ret (alias for jr r14)
```

A function that changes `r5-r12` must restore them before returning. A function that uses `r14` for a nested call must save and restore it. The ordinary function ABI and Linux syscall ABI are separate conventions.

4. Linux syscall ABI

```
r1       syscall number on entry; signed result on return
r2-r7    syscall arguments, in order
syscall 0  conventional immediate field; Linux number is still in r1
```

Common currently exercised numbers:

Number	Call
29	ioctl
56	openat
57	close
63	read
64	write
74	fsync
93/94	exit / exit_group
160	uname
214	brk
220	clone
221	execve
260	wait4

Raw result: `r1 >= 0` is success; `r1 < 0` is `-errno`. Libc wrappers translate this to `-1` plus `errno`; hand-written assembly must not apply that translation twice.

5. Current REM mnemonics

```
Load immediate       li   r1, value
Add                  add  rd, ra, rb
Add immediate        addi rd, ra, imm
Subtract             sub  rd, ra, rb
Logic                and  or  xor  not
Shift                sll  srl  sra
Compare              seq  sne  slt  sge  sltu  sgeu
Load/store           lw   rd, displacement(base)
                     sw   rs, displacement(base)
Branch               beq  bne  blt  bge  bltu  bgeu
Jump                 j label
Call/link            jal label
Register jump        jr r14
Return               ret
System call          syscall 0
Stop                 halt
```

Historical `MOV`, `LOAD`, `STORE`, `JMP`, `CALL`, and `RET` examples are explanatory or legacy syntax. See Appendix L of *Inside the REM Computer*.

6. ELF and inspection

```
source .S
   ↓ myemulator2-elf-as
object .o
   ↓ myemulator2-elf-ld + linker script
ELF32 little-endian executable
```

```
TOOL=.toolchain-install/bin/myemulator2-elf
$TOOL-as main.S -o main.o
$TOOL-ld -T toolchain/userspace/myemulator2-user.ld main.o -o program.elf
$TOOL-readelf -h -l -S -s -r program.elf
$TOOL-objdump -dr program.elf
$TOOL-nm program.elf
$TOOL-objcopy -O binary program.elf program.bin
```

Use `readelf -h` for the machine and entry point, `-l` for loadable segments, `-r` for unresolved relocations, and `objdump -dr` to correlate source with machine code.

7. QEMU debugging

```
.qemu-build/qemu-system-myemulator32 \
  -machine myemulator32 -kernel program.elf \
  -nographic -d in_asm,cpu,nochain -D qemu.log
```

Read the last `PC` before the fault. Map it to:

```
$TOOL-objdump -dr program.elf
```

For a paused host-side debug stub:

```
.qemu-build/qemu-system-myemulator32 \
  -machine myemulator32 -kernel program.elf \
  -S -gdb tcp::1234
```

Trace fields:

```
PC       instruction address
OBJD-T   raw instruction word printed by this QEMU target
R1...    register state before the instruction
SR       status and privilege state
```

If the address is not in a loadable segment, inspect ELF program headers and the linker script before changing the CPU decoder.

8. Terminal bytes

```
ESC = 0x1B
CSI = ESC [ = 1B 5B

Clear screen       ESC [ 2 J       1B 5B 32 4A
Cursor home        ESC [ H         1B 5B 48
Move row 10 col 20 ESC [ 10 ; 20 H
Erase line right   ESC [ 0 K
Erase line left    ESC [ 1 K
Erase entire line  ESC [ 2 K
Reset attributes   ESC [ 0 m
Hide cursor        ESC [ ? 25 l
Show cursor        ESC [ ? 25 h
```

Arrow input in the usual application mode:

```
Up       ESC [ A       1B 5B 41
Down     ESC [ B       1B 5B 42
Right    ESC [ C       1B 5B 43
Left     ESC [ D       1B 5B 44
```

After an isolated `ESC`, wait briefly before deciding whether it is the Escape key or the beginning of a sequence. Keep terminal bytes in the terminal module; pass decoded key events to the editor.